
```
def displayTowers(towers):
    """Display the three towers with their disks."""

    # Display the three towers:
    for level in range(TOTAL_DISKS, -1, -1):
        for tower in (towers["A"], towers["B"], towers["C"]):
            if level >= len(tower):
                displayDisk(0) # Display the bare pole with no disk.
            else:
                displayDisk(tower[level]) # Display the disk.
        print()
```

It relies on the `displayDisk()` function, which we'll cover next, to display each disk in the tower. The `for level` loop checks every possible disk for a tower, and the `for tower` loop checks towers A, B, and C.

The `displayTowers()` function calls `displayDisk()` to display each disk at a specific width, or if `0` is passed, the pole with no disk:

```
# Display the tower labels A, B, and C:
emptySpace = ' ' * (TOTAL_DISKS)
print('{0} A{0}{0} B{0}{0} C\n'.format(emptySpace))
```

We display the A, B, and C labels onscreen. The player needs this information to distinguish between the towers and to reinforce that the towers are labeled A, B, and C rather than 1, 2, and 3 or Left, Middle, and Right. I chose not to use 1, 2, and 3 for the tower labels to prevent players from confusing these numbers with the numbers used for the disks' sizes.

We set the `emptySpace` variable to the number of spaces to place in between each label, which in turn is based on `TOTAL_DISKS`, because the more disks in the game, the wider apart the poles are. Rather than use an f-string, as in `print(f'{emptySpace} A{emptySpace}{emptySpace} B{emptySpace}{emptySpace} C\n')`, we use the `format()` string method. This allows us to use the same `emptySpace` argument wherever `{0}` appears in the associated string, producing shorter and more readable code than the f-string version.

The `displayDisk()` function displays a single disk along with its width. If no disk is present, it displays just the pole: